GenAI FDE Production Lab

Supporting Architecture, Setup, and Interview Guide

By Tobias Weissmann

Purpose: This document supports the companion GitHub repository and explains how candidates can use it to demonstrate production-aware GenAI Forward Deployed Engineer reasoning in interviews.

Item	Description
Repo name	genai-fde-production-lab
Primary audience	GenAI FDE / AI Engineer / FDSE candidates
Core proof	Working vertical slice for enterprise RAG, HITL, evaluations, observability, MCP, A2A, and text-to-SQL safety
Positioning	Production-aware educational lab, not a drop-in enterprise platform

1. Product Positioning

The repository is designed to help candidates move from theory to proof. It lets them say: I built and can explain a production-aware enterprise GenAI system.

The lab should be positioned honestly. It is not a complete enterprise SaaS platform. It is a compact, explainable system that demonstrates key production patterns: multi-tenant retrieval, provider abstraction, approval workflows, quality gates, observability, and safe tool execution.

2. Architecture Overview

Users interact with a React frontend. FastAPI resolves tenant context, roles, permission scopes, and request identity. The RAG orchestrator retrieves permission-filtered chunks, routes generation through the LLM provider layer, then sends safe answers directly or risky actions into a human review queue.

Background ingestion jobs sync source data, chunk documents, create embeddings, and index chunks. Evaluation jobs and incident simulations validate quality, safety, and operational readiness.

3. Repository Structure

Folder	Purpose
backend/	FastAPI app, auth, connectors, ingestion, RAG, HITL, evals, incidents, A2A, text-to-SQL, tests
frontend/	React workflow shell for chat, review queue, admin dashboard, incidents, and eval runs
mcp_server/	Safe MCP tool examples with schemas, validation, tenant checks, and audit contracts
deploy/	Docker, Kubernetes, Terraform, Prometheus, Grafana, and OpenTelemetry examples
docs/	Architecture, interview guide, security model, RAG design, HITL workflow, MCP, A2A, and text-to-SQL docs
scripts/	Seed data, local eval runner, and incident demo entrypoints

4. Core Vertical Slice

- Seed demo documents for a tenant.
- Run incremental sync and create document chunks.
- Generate deterministic mock embeddings.
- Retrieve chunks only for the current tenant and permission scopes.
- Generate an answer with citations.
- Classify risky actions with approval policy.
- Create a human review request.
- Approve, edit, reject, or escalate from the review queue.
- Record every decision in the audit log.
- Run evaluation quality gates.

5. Human-in-the-Loop Workflow

The HITL workflow follows this interview-ready framework:

AI Suggests -> Policy Evaluates -> Human Reviews -> Approves / Edits / Rejects / Escalates -> System Executes -> Audit Log Records -> Evaluation Updates

State	Meaning
PENDING_REVIEW	AI proposal is waiting for a reviewer decision.
APPROVED	Reviewer accepts the proposed action.

APPROVED_WITH_EDITS	Reviewer modifies output before approval.
REJECTED	Reviewer blocks the action.
ESCALATED	Reviewer sends the decision to a higher authority.
ROLLED_BACK	Executed action required rollback or remediation.

6. RAG and Permission Model

Area	Design
Tenant isolation	Every document and chunk carries tenant_id. Retrieval must filter by tenant before context assembly.
Permission scopes	Chunks carry source permission scopes such as public, support, sales, legal, or compliance.
Deletion handling	Deleted source items must be removed or hidden from retrieval.
Stale-index risk	Sync lag and content hash mismatch are monitored and simulated.

7. Evaluation and CI Gates

Area	Design
RAG quality	Minimum score threshold blocks low-quality releases.
Permission safety	Cross-tenant leakage must remain zero.
Safety evals	Unsafe action and prompt-injection cases are tested.
SQL safety	Mutation queries are blocked and LIMIT is required.

8. Observability and Incidents

Area	Design
Latency spike	Detect p95 latency, mitigate with timeouts, fallback, and degraded mode.
Permission leak	Detect tenant isolation failure, disable affected index, reindex, audit impact.
Stale index	Detect sync lag and content hash mismatch, force refresh.
Cost spike	Detect token budget increase, reduce chunks and route cheaper model.

9. MCP and A2A

Area	Design
MCP tools	Expose safe tools for document search, review creation, safe SQL, and ticket context.
Tool safety	Every tool requires schema validation, tenant check, permission check, and audit log.
A2A messages	Agents preserve request_id, trace_id, tenant_id, sender, receiver, payload, and risk level.
Safety boundary	Agents must never bypass policy, retrieval, or approval controls.

10. Text-to-SQL Design

Area	Design
Schema index	Select only relevant tables to reduce prompt size and risk.
Validator	Allow SELECT only; block INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, and similar operations.
Sandbox	Execute with timeout, LIMIT, row limits, and tenant filters.
Approval gate	Sensitive or high-risk queries require reviewer approval.

11. Interview Talking Points

Answer Type	Example

Weak	I built a RAG chatbot.
Strong	I built a permission-aware RAG workflow where tenant and source permissions are enforced before LLM context assembly.
Weak	I call an LLM provider from the backend.
Strong	I use a provider interface so latency, cost, token usage, fallback, and mock testing are separated from application logic.
Weak	A human can review the output.
Strong	Risk policy creates structured review requests with evidence, confidence, override reasons, escalation states, and audit logs.
Weak	I run tests.
Strong	CI runs unit tests plus quality gates for RAG, permission leakage, safety, and SQL validation.

12. Practical Build Roadmap

Milestone	Scope
MVP	Working local mock provider, seed data, permission-aware retrieval, HITL queue, audit log, eval gate, tests.
V1.1	Real pgvector implementation, real OpenAI provider call, richer review UI, more eval datasets.
V1.2	OpenTelemetry metrics, Prometheus dashboard, Grafana dashboard, incident scripts.
V1.3	MCP server tools, A2A workflow, text-to-SQL sandbox, connector improvements.
V2.0	More realistic deployments, Terraform examples, Kubernetes hardening, interview demo scripts.

13. Buyer-Value Explanation

The repo makes the paid GenAI FDE product feel more practical and premium because it gives buyers a portfolio asset, not only reading material. It supports architecture discussions, code walkthroughs, incident practice, and system-design storytelling.

The most important commercial promise is credibility: candidates can explain not only what GenAI systems should do, but also how production constraints change the design.

14. Recommended Disclaimer

Use this repository as a production-aware educational lab. Do not claim it is a complete enterprise platform. The value is in its architecture, safety controls, interfaces, tests, evaluation gates, and interview explanations.